

Reviewer Pack — Minimal Diophantine Root Count and Frege Proof Tree Width In...

Entry c18380f06df9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 07:22:27 UTC

Minimal Diophantine Root Count and Frege Proof Tree Width Inequality

Entry ID: c18380f06df9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 07:22:27 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Geometry) **Field B** (complexity object): Resolution Proofs (Frege Proof Complexity)

Statement:

For every CNF φ , the minimal number of distinct algebraic integers that are roots of at least one polynomial with coefficients in $\mathbb{Z}/2\mathbb{Z}$ associated with φ is linearly correlated with its Frege proof width $w(\varphi)$, such that $|R(\varphi)| = \Theta(w(\varphi))$

Rationale (proposer's reasoning):

Diophantine geometry, through the lens of algebraic integers, can encode logical relationships within a CNF. The roots of these polynomials could potentially reflect the complexity of proving unsatisfiability in φ , providing a new perspective on proof width.

Taxonomy category: Frege Proof Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a2af3ca326c17ee4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each CNF φ , if the absolute difference between the minimal number of distinct algebraic integers $|R(\varphi)|$ and its Frege proof width $w(\varphi)$ is within a factor of 3 for at least 24 out of 30 seeds, or if any seed produces an $|R(\varphi)|$ outside the range $[0.5 * w(\varphi), 1.5 * w(\varphi)]$, then the conjecture is supported or falsified, respectively.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def frege_proof_width(phi):
        # Placeholder implementation for Frege proof width calculation
        return len(phi.split())

    def find_algebraic_roots(phi):
        # Placeholder implementation for finding algebraic roots
        # This is a dummy function and should be replaced with actual logic
        return set([random.randint(1, 10) for _ in range(len(phi.split()))])

    phi = " ".join(random.choices("01", k=random.randint(5, 20)))
    w_phi = frege_proof_width(phi)
    R_phi = find_algebraic_roots(phi)

    return {
        "metric_name": "R( $\phi$ ) - w( $\phi$ )",
        "metric_value": abs(len(R_phi) - w_phi),
        "instances_tested": 1,
        "n_max": len(phi.split()),
        "conjecture_holds": abs(len(R_phi) - w_phi) <= 3 * w_phi / 2,
        "counterexample": "" if conjecture_holds else f"phi={phi}, R( $\phi$ )={len(R_phi)}, w( $\phi$ )={w_phi}"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)
```

```

mean_value = sum(result["metric_value"] for result in results) / len(results)
std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)][\"counterexample\"]}\"")
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_999188e7.py", line 48, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_999188e7.py", line 40, in run_trial
    "counterexample": "" if conjecture_holds else f"phi={phi}, R(φ)={len(R_phi)}, w(φ)={w_phi}"
                    ^^^^^^^^^^^^^^^^^^^^^^^^^
NameError: name 'conjecture_holds' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23644
2	preregistration	ollama_remote	glm4:latest	0	0	11509
3	novelty	ollama_remote	glm4:latest	0	0	8704
4	novelty	ollama_remote	glm4:latest	0	0	8755
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14775

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7923
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8076
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6679
9	judge	ollama_remote	glm4:latest	0	0	14657

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104721 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/c18380f06df9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c18380f06df9.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c18380f06df9.tar.gz (if generated)